

Engenharia de Software

Profa. Dra. Débora Maria Barroso Paiva



Módulo 4 - Teste e manutenção de software

Unidade 1 - Estratégias de teste de software



Objetivo

- Entender as principais técnicas usadas para testar software e realizar sua manutenção, incluindo atividades de reengenharia de software.

Verificação e validação de software

- **Validação:** Estamos construindo o produto correto?
 - assegurar que o software atende às expectativas dos clientes.
- **Verificação:** Estamos construindo o produto corretamente?
 - verificar se o software está de acordo com suas especificações.

Verificação e validação de software

- Incluem várias atividades, tais como:
 - Revisões técnicas ou inspeções de software;
 - Auditoria;
 - Simulação;
 - Testes;
 - Etc.

Verificação e validação de software

- Inspeções de software:
 - analisam e verificam representações de sistemas, como documento de requisitos, diagramas de projeto e código-fonte de programa.
- Testes de software:
 - envolvem executar uma implementação do software com dados de teste.

Verificação e validação de software

- O objetivo principal da atividade de teste é ***mostrar a presença de defeitos***, caso eles existam (realizar testes não garante que um software está correto).
- Quando os testes são realizados de forma criteriosa, o que se tem é uma certa ***confiança*** de que ele se comporta corretamente.

Como fazer inspeção de software?

- É um processo formal realizado por uma equipe de pelo menos quatro pessoas;
- Os membros da equipe analisam o artefato e apontam defeitos;
- Papéis: autor, leitor, testador e moderador;
- O leitor lê em voz alta para a equipe de inspeção, o testador inspeciona o artefato e o moderador organiza o processo.

Inspeção de software

- Como resultado de uma inspeção, diversos problemas podem ser encontrados:
 - defeitos de controle;
 - defeitos de gerenciamento de exceções;
 - inconsistência;
 - informação incorreta;
 - etc.

Teste de software

- Relembrando:

Processo de executar um programa com o objetivo de revelar a presença de defeitos, ou, falhando nesse objetivo, aumentar a confiança sobre o programa

Estratégia de teste de software

- Qualquer estratégia de teste deve incorporar:
 - planejamento de teste;
 - projeto de casos de teste;
 - execução de testes;
 - a resultante coleta e avaliação de dados.

Caso de teste

- Um **caso de teste** é um par ordenado no qual o primeiro elemento representa uma entrada para o programa, e o segundo, a correspondente saída esperada.
- **Entrada:** conjunto de dados necessários para uma execução do programa;
- **Saída esperada:** resultado de tal execução do programa.

Caso de teste - exemplo

- Considere um programa que recebe como parâmetros de entrada dois números inteiros, x e y , com $y \geq 0$ e computa x^y .
- Casos de teste para o exemplo são:
 - $\langle (2,3), 8 \rangle$
 - $\langle (-4,3), -64 \rangle$
 - $\langle (3,-1), \text{Erro} \rangle$

Casos de teste em Desenvolvimento Dirigido por Testes (TDD)

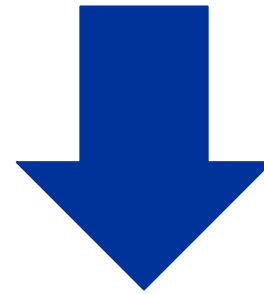
- Em TDD é possível elaborar os casos de teste enquanto cartões de estória são escritos.

Casos de teste em Desenvolvimento Dirigido por Testes (TDD)

- O teste é escrito antes da tarefa ser implementada;
- Após a implementação do software, o teste pode ser colocado em prática imediatamente;
- O desenvolvimento *test-first* e o uso de ferramentas de teste automatizadas são os principais pontos fortes da abordagem XP.

Casos de teste

- O projeto dos casos de teste pode ser tão difícil quanto o projeto do próprio produto a ser testado.



**Técnicas de Projeto de
Casos de Teste**

Casos de teste

- Os casos de teste devem levar em consideração os requisitos;
- Os testes devem ser planejados com antecedência, desde o início do desenvolvimento;
- Um caso de teste deve levar em consideração não só o conjunto de dados de entrada, mas também a saída esperada;

Casos de teste

- Um conjunto de testes deve exercitar não só as condições válidas, mas também as inválidas e inoportunas;
- Casos de teste devem ser guardados para que possam ser replicados;
- Um bom caso de teste é aquele que tem alta probabilidade de encontrar uma falha que não havia sido encontrada ainda.

Fases de teste

- O processo de teste inicia com os testes de unidade, que visam verificar se cada módulo ou unidade do software satisfaz à sua especificação;
- Após testar separadamente cada módulo, estes são agrupados para compor os subsistemas, sendo esta a fase de testes de integração;

Fases de teste

- Uma vez que os subsistemas são testados isoladamente, estes são integrados formando o software completo. Neste momento começa a fase de **testes de sistema**, cujo objetivo é exercitar o sistema como um todo;
- Finalmente, os **testes de aceitação** visam validar o software, ou seja, determinam se ele satisfaz aos requisitos especificados na fase de especificação do sistema.

Fases de teste

- Teste de Unidade:
 - Identificar erros de lógica e de implementação em cada módulo do software, separadamente.
- Teste de Integração:
 - Identificar erros associados às interfaces entre os módulos do software.

Fases de teste

- Teste de Sistema:
 - Verificar se as funções estão de acordo com a especificação e se todos os elementos do sistema combinam-se adequadamente.
- Teste de Aceitação:
 - Realizado pelos usuários finais.

Teste de unidade

- Focaliza o esforço de verificação na menor unidade de projeto do software.
- Testa os caminhos de controle nos componentes.
- **Foco principal:** lógica interna de processamento e as estruturas de dados .

Teste de unidade

- Erros mais comuns:
 - Precedência aritmética mal entendida ou incorreta;
 - Inicialização incorreta;
 - Falta de precisão;
 - Comparação de tipos de dados diferentes;
 - Operadores ou precedência lógica incorretos;

Teste de unidade

- Erros mais comuns:
 - Comparação incorreta de variáveis;
 - Falhas na saída;
 - Descrição do erro ininteligível;
 - Erro mencionado não corresponde ao erro encontrado;
 - Erros nos limites.

Teste de unidade

- Em situações em que não existam recursos para fazer testes de unidade abrangentes: escolha os módulos (componentes) críticos;
- Um teste de unidade é simplificado quando um componente com alta coesão é projetado (apenas uma função é implementada por um componente).

Teste de integração

- O objetivo é, a partir dos módulos testados ao nível de unidade, construir a estrutura de programa que foi determinada pelo projeto;
- Os módulos são montados ou integrados para formarem um pacote de software.

Teste de integração

- Verifica se os componentes funcionam realmente em conjunto, se são chamados corretamente e se transferem dados corretos no tempo correto por meio de suas interfaces.
- **Integração não-incremental:** o programa completo é testado como um todo e o resultado é o caos.

Teste de integração

- **Integração incremental:**

- o programa é construído e testado em pequenos segmentos, nos quais os erros são mais fáceis de serem isolados e corrigidos;
- as interfaces têm maior probabilidade de ser testadas completamente;
- uma abordagem sistemática ao teste pode ser aplicada.

Teste de sistema

- O software é apenas um elemento de um sistema baseado em computador mais amplo.
- O teste de sistema envolve uma série de diferentes testes, cujo propósito primordial é pôr completamente a prova o sistema baseado em computador.

Exemplos - teste de sistema

- Forçar o software a falhar de diversas maneiras e verificar se a recuperação é adequadamente executada → *(teste de recuperação)*.
- Verificar se todos os mecanismos de proteção embutidos num sistema o protegerão, de fato, de acessos indevidos → *(teste de segurança)*.

Exemplos - teste de sistema

- Executar o sistema de uma forma que exija recursos em: quantidade, frequência ou volume anormais → *(teste de estresse)*
 - A velocidade de entrada de dados pode ser aumentada;
 - Casos de teste que exigem o máximo de memória podem ser executados;
 - Casos de teste que podem causar busca excessiva de dados residentes em disco são criados.

Teste de aceitação

- Ao término da atividade de teste, o software está completamente montado como um pacote.
- Diferentes erros foram descobertos e corrigidos.
- Uma série final de testes de software, (*testes de aceitação*), podem iniciar.

Teste de aceitação

- O plano e o procedimento de teste são projetados para garantir que:
 - todos os requisitos funcionais são satisfeitos;
 - todos os requisitos de desempenho são obtidos;
 - a documentação está correta;
 - outros requisitos são cumpridos: portabilidade, compatibilidade, remoção de erros e manutenibilidade.

Considerações finais

- **Teste de software** é uma atividade fundamental do processo de desenvolvimento de software e está diretamente relacionado à qualidade dos sistemas que são entregues.
- Há diferentes abordagens de teste, que são colocadas em prática de acordo com os recursos disponíveis em cada empresa.

Referências



AGEAD
Agência de Educação
Digital e a Distância



PRESSMAN, Roger S; MAXIM, Bruce R. **Engenharia de software**: uma abordagem profissional. 9. ed. Porto Alegre: McGraw Hill, 2021. ISBN 9786558040118. [Disponível na Biblioteca Digital da UFMS.](#)

SOMMERVILLE, Ian. **Engenharia de software**. 10. ed. São Paulo: Editora Pearson, 2018. ISBN 9788543024974. [Disponível na Biblioteca Digital da UFMS.](#)

Licenciamento



Respeitadas as formas de citação formal de autores de acordo com as normas da ABNT NBR 6023 (2018), a não ser que esteja indicado de outra forma, todo material desta apresentação está licenciado sob uma [Licença Creative Commons - Atribuição 4.0 Internacional](https://creativecommons.org/licenses/by/4.0/).